

Agentic Middleware

다중 LLM 에이전트 트랜잭션을 위한 QCFuse + ATCC 기반 미들웨어

Problem

LLM reasoning이 길어질수록
DB lock과 rollback 비용이 증폭

Solution

QCFuse-style read fusion +
ATCC-style cost-aware arbitration

Demo

Go gRPC middleware
+ Python agents

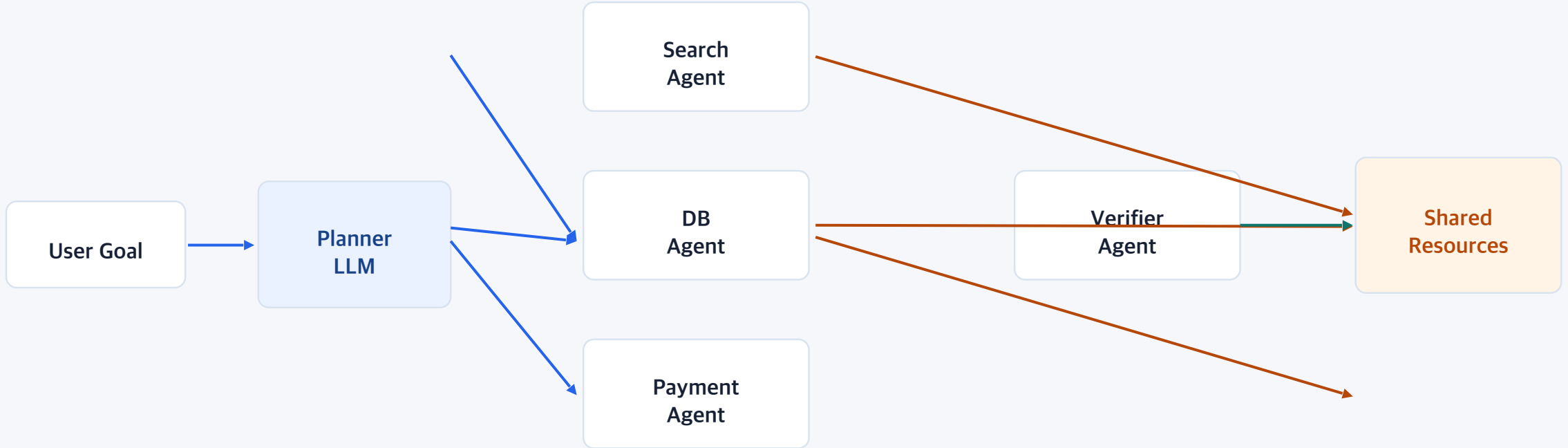
발표 흐름

문제 정의에서 데모까지, 평가자가 따라오기 쉬운 구조로 구성

- | | | |
|----|----------------------------|--|
| 01 | Agentic AI 시대의 workload 변화 | 단일 호출에서 다중 sub-agent 동시 실행으로 이동 |
| 02 | 기존 트랜잭션 제어의 한계 | 긴 reasoning, lock 대기, rollback 비용 증폭 |
| 03 | 관련 연구와 gap | SagaLLM, ATCC, QCFuse, AIOS의 역할 정리 |
| 04 | 제안 아키텍처와 구현 | Python agent layer + Go middleware layer |
| 05 | 실험/데모/한계 | 지표 기반 결과와 최종평가 데모 흐름 |

연구 배경: Agentic AI는 동시 실행 구조로 이동한다

하나의 LLM 응답이 아니라, 여러 agent가 계획·도구호출·검증을 병렬 수행하는 구조



동시성은 agent의 성능을 높이지만,
공유 자원 접근 시 transaction conflict를 만든다.

문제 정의: Agentic transaction은 기존 OLTP와 다르다

짧고 예측 가능한 transaction이 아니라, 긴 reasoning과 비결정적 접근을 포함한다

구분	기존 OLTP transaction	Agentic transaction
실행 시간	수 ms~수십 ms	LLM reasoning으로 수 초~수 분
접근 패턴	쿼리/락 범위 예측 가능	도구 호출 결과에 따라 동적으로 변화
Abort 비용	DB 작업 일부 손실	Token/API/reasoning latency 전체 손실
충돌 처리	무작위 abort도 비용 작음	고비용 agent abort 시 경제적 손실 큼

핵심 질문: 여러 agent가 같은 자원에 접근할 때, 시스템은 어떤 요청을 보호하고 어떤 요청을 rollback해야 하는가?

관련 연구: transaction 보장, scheduling, cache fusion은 각각 분리되어 있다

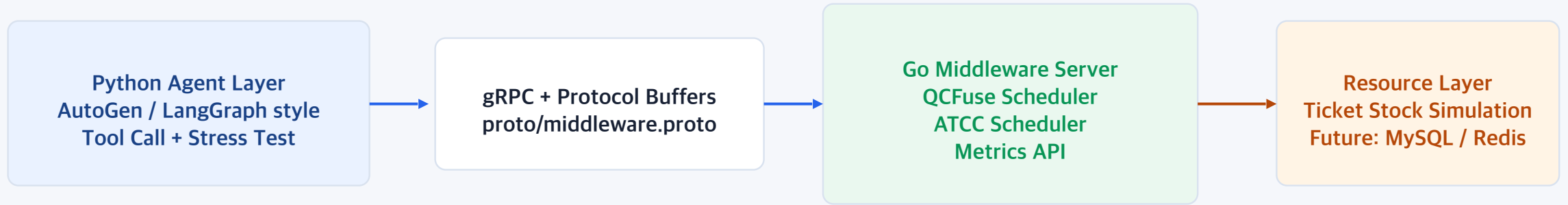
본 프로젝트는 세 흐름을 agentic middleware layer에서 결합한다

Concurrent Modular Agent	다중 LLM 모듈의 비동기 병렬 실행	공유 DB 충돌/rollback cost는 직접 해결하지 않음
AIOS	Agent OS 관점의 scheduler/tool/memory 관리	DB transaction guard는 별도 구현 필요
SagaLLM	LLM workflow에 Saga compensation 도입	동시 경합 시 비용 기반 rollback 우선순위는 부족
ATCC	Agentic transaction의 abort cost-aware CC	본 프로젝트는 token/latency 기반 단순화 구현
QCFuse	Query-centric KV cache fusion	본 프로젝트는 read request fusion으로 재해석

Gap: 다중 agent transaction에서 read amplification + lock wait + sunk-cost rollback을 함께 다루는 middleware

제안 아키텍처: Python Agent 계층과 Go Middleware 계층 분리

gRPC typed contract를 기준으로 agent runtime과 transaction control을 분리



Phase 1 ReadResource

동일 자원 조회 요청을 짧은 window에 모아 fusion

Phase 2 Lock-Free Reasoning

LLM reasoning 동안 DB connection/lock을 점유하지 않음

Phase 3 CommitTransaction

token + latency 기반 sunk cost로 winner 선정

QCFuse-style Read Fusion: 중복 조회를 하나의 I/O로 압축

원 논문의 KV cache fusion 아이디어를 middleware read path의 request fusion으로 재해석



Deterministic Demo

5 → 1

5개의 read를 1개 batch로 융합

Saved DB Reads

4

동일 자원 조회에서 줄인 I/O

Stress Target

10/50/100/200

최종보고서용 확장 실험

Lock-Free Reasoning: LLM 추론 시간은 DB lock으로 전이되지 않아야 한다

Agent는 조회 후 독립적으로 reasoning하고, commit 시점에만 middleware arbitration에 참여



발표 메시지

LLM이 생각하는 동안 DB lock을 붙잡지 않는다. 긴 reasoning은 agent layer에서 진행하고, conflict resolution은 commit phase에서만 처리한다.

ATCC-style Cost-Aware Arbitration: 매몰 비용이 큰 agent를 보호

원 ATCC의 abort-cost-aware priority scheduling을 token/latency 기반 score로 단순화

Sunk Cost Score

$$\text{cost} = \text{tokens} \times 0.002 + \text{latency_sec} \times 0.5$$

Commit Rule

경합 batch를 score 내림차순 정렬 → 1등 commit, 나머지는 rollback signal

1	Agent-A	\$13.25	WIN
2	Agent-D	\$8.75	ROLLBACK
3	Agent-C	\$5.10	ROLLBACK
4	Agent-E	\$2.85	ROLLBACK
5	Agent-B	\$1.00	ROLLBACK

시스템 구현: 최소하지만 검증 가능한 middleware prototype

Go channel scheduler, gRPC contract, Python demo runner, metrics dashboard

Go Middleware

QCFuseScheduler / ATCCScheduler
metrics endpoint / config env

Protocol Buffers

ReadResource / CommitTransaction
Python-Go typed contract

Python Agent

deterministic_demo.py
stress_test_v2.py / AutoGen scenario

Dashboard

live metrics / leaderboard
video recording script

핵심 원칙: agent framework 내부가 아니라 middleware layer에서 공통 transaction guard를 제공한다.

LLM Tool Call 안정화: hallucination을 실행 가능한 contract로 제한

AutoGen/LangGraph 계층에서 발생할 수 있는 파라미터 누락·타입 오류·도구 미호출 문제 대응

문제 1

LLM이 token_cost를 문자열로 보내거나 필드를 누락

Type Hint + Docstring

tool schema를 명확히 하여 LLM이 올바른 JSON argument를 구성하도록 유도

문제 2

도구를 호출하지 않고 자연어로만 '결제 완료'라고 응답

Canonical Proto

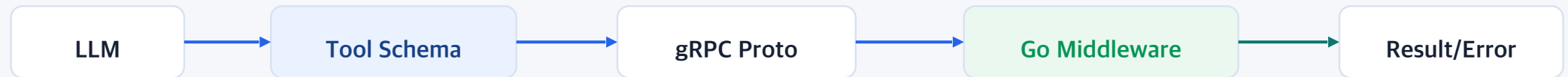
Python/Go 사이 필드명과 타입을 하나의 proto 계약으로 고정

문제 3

middleware 통신 에러가 agent workflow 전체 crash로 이어짐

Self-healing Message

통신 에러를 문자열로 반환해 agent가 재시도/수정할 수 있게 함



실험 설계: 동일 자원 재고 1장에 대한 다중 agent 경합

주차 발표에서 사용한 10/50/100/200 agent stress 시나리오를 최종평가용으로 정리

Resource	Agents	Reasoning	Cost
1 ticket	10~200	2~10s	\$ proxy
flight_ticket_A	동시 접속 thread	LLM latency simulation	token + latency

동시 진입: 모든 agent가 동일 resource 조회
QCFuse: 짧은 window에서 read request fusion
Lock-Free: agent별 reasoning latency simulation
ATCC: commit batch leaderboard 정렬
결과: winner commit, loser rollback signal

측정 지표: saved DB reads, commit/rollback 수, total saved cost, elapsed time, throughput

Deterministic Demo Result: 발표 중 항상 같은 결과를 재현

5명의 agent를 고정 비용으로 실행하여 dashboard와 JSON/CSV 결과를 함께 보여준다

Winner

Agent-A

highest sunk cost

Rollbacks

4

Saga-style rollback

Saved DB Reads

4

QCFuse read fusion

Saved Cost

\$17.70

ATCC rollback value

```
cd middleware-go && GOCACHE=/private/tmp/agenic-middleware-gocache go run .  
cd agent-python && python3 deterministic_demo.py  
open dashboard.html
```

발표 포인트: 단순 성공/실패가 아니라, middleware가 절약한 I/O와 보호한 비용을 숫자로 보여준다.

Stress Test 관찰 결과: agent 수가 늘어도 rollback/error는 제어 가능

12주차 자료의 10/50/100/200 thread 실험 결과를 최종평가용으로 재정리

Agents	처리 시간	Success	Rollback	Error
10	~10 sec	3	7	0
50	~10 sec	4	46	0
100	~10 sec	4	96	0
200	~10 sec	4	196	0

해석

재고 1장이지만 ATCC window 단위로 batch가 반복되며 각 window마다 1건 승인된다. 최종보고서에서는 window 설정과 재고 reset 정책을 명확히 설명한다.

경제적 효과: 무작위 rollback보다 고비용 agent를 보호한다

12주차 50명 경합 분석의 핵심 메시지를 비용 관점으로 정리

Random-like Loss

\$405.74

무작위/최악 승인 시 손실 예시

ATCC Loss

\$374.46

비용 기반 승인 적용

Protected Value

\$31.28

약 4.2만원 상당

핵심 주장

Agentic workload에서 rollback은 단순 DB 작업 취소가 아니라 이미 소모한 LLM token/API/reasoning time의 폐기다. 따라서 conflict resolution은 비용을 인지해야 한다.

주의: 최종보고서에서는 simulation 조건과 cost proxy임을 명확히 표시

데모 시연: Dashboard에서 middleware의 결정 과정을 실시간으로 확인

교수님께 보여줄 영상/실시간 시연 흐름

1

Go middleware 실행

gRPC :50051 / metrics :8080

2

dashboard.html 열기

live 상태와 metric 초기값 확인

3

deterministic_demo.py 실행

5 agents read + reasoning + commit

4

결과 설명

Agent-A winner, rollback 4, saved reads 4, saved cost \$17.70

영상 포인트: terminal command → dashboard metric 변화 → JSON/CSV 결과 파일 확인

프로젝트 기여와 한계

완성된 prototype의 의미와 최종평가 이후 확장 방향

기여

Agentic transaction 문제를 token/API/reasoning cost 관점으로 재정의
QCFuse-style read fusion과 ATCC-style arbitration을 하나의 middleware로 결합
Python agent layer와 Go middleware layer를 gRPC contract로 분리
Metrics API와 dashboard로 middleware 결정 과정을 시각화

한계와 향후 계획

현재 resource layer는 DB simulation이며 실제 MySQL/Redis 연동은 향후 과제
Cost formula는 token/latency proxy 기반의 단순화 모델
ATCC 원 논문의 RL 기반 phase-aware policy는 구현 범위 밖
AutoGen/LangGraph full workflow는 optional scenario로 추가 안정화 필요

결론

Agentic AI 환경에서 트랜잭션 제어는
정합성뿐 아니라 비용 보존의 문제가 된다.

QCFuse-style

중복 read I/O를 fusion

Lock-Free

reasoning 동안 DB lock 해제

ATCC-style

매물 비용 기반 commit arbitration

Q & A

GitHub: <https://github.com/singsangssong/graduation-project>